

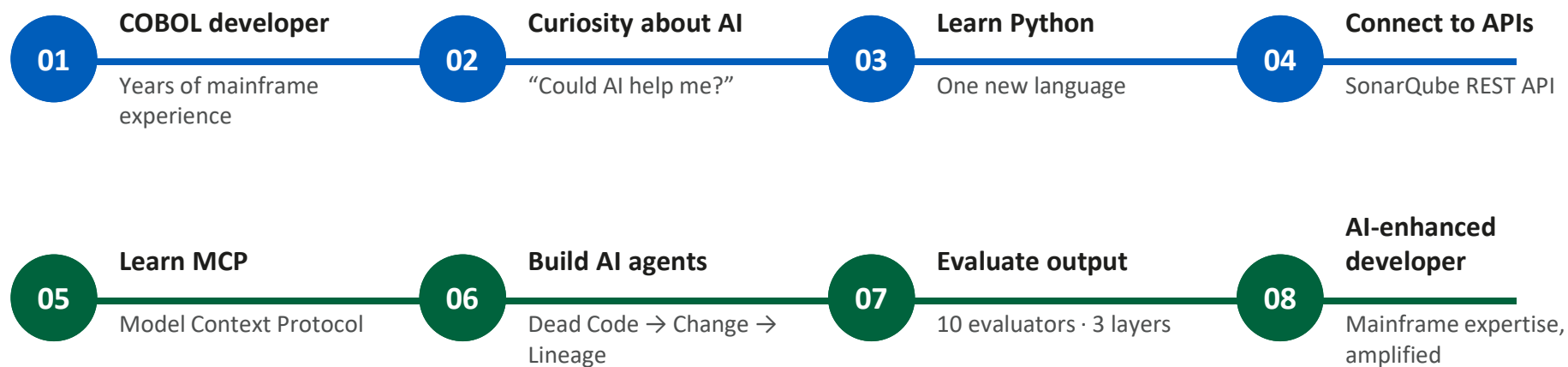


Agentic AI for Enterprise Mainframes: From Dead Code Elimination to Business Knowledge

.

Thamarai Selvi Ravi Kumar
Mainframe Engineer · Legal & General

One Step at a Time



THREE AGENTS BUILT ALONG THE WAY

Dead Code Agent

Remove unused code safely

Code Change Agent

Generate approved changes

Lineage Agent

Trace COBOL → DB2 lineage

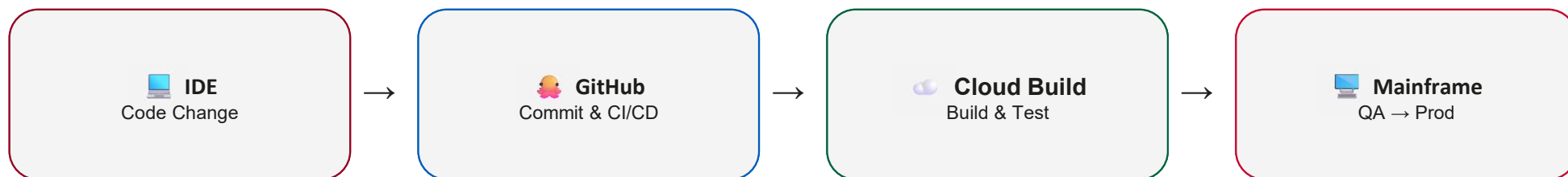
Mainframe CI/CD

From Mainframe to Cloud and Back — the delivery infrastructure our agents operate within

SOURCE OF TRUTH — KEPT ALIGNED VIA GIT-MAINFRAME SYNC



DELIVERY FLOW



THE PROBLEM

Understanding Legacy Applications

Dead code is more than clutter — it has concrete consequences



Slows Innovation

Developers must sift through irrelevant logic before making any change



Hides Business Logic

Real business rules become buried under layers of unused code



Misleads AI Tools

AI tools misinterpret the system when they process dead code as active logic



The Insight

“We had code that appeared to be unused, but removing code from a large mainframe application requires confidence.”

AGENT #1

Dead Code Agent

An agent-based pipeline: from static analysis findings to safe code remediation



MCP in Simple Terms

“MCP became a bridge that allowed the AI agent to use tools and information from the dev environment — like a universal adapter.”

The AI Did Not Get a Production Keyboard

AI recommends. Humans approve. AI executes. Humans verify.

1 Dead Code Analysis Complete

2 AI Recommendation Generated

3 Human Reviews Recommendation

👉 **APPROVAL GATE — Human Decision**

4 Code Change Agent Executes

5 Human Reviews Final Change

✗ Before — Manual Process

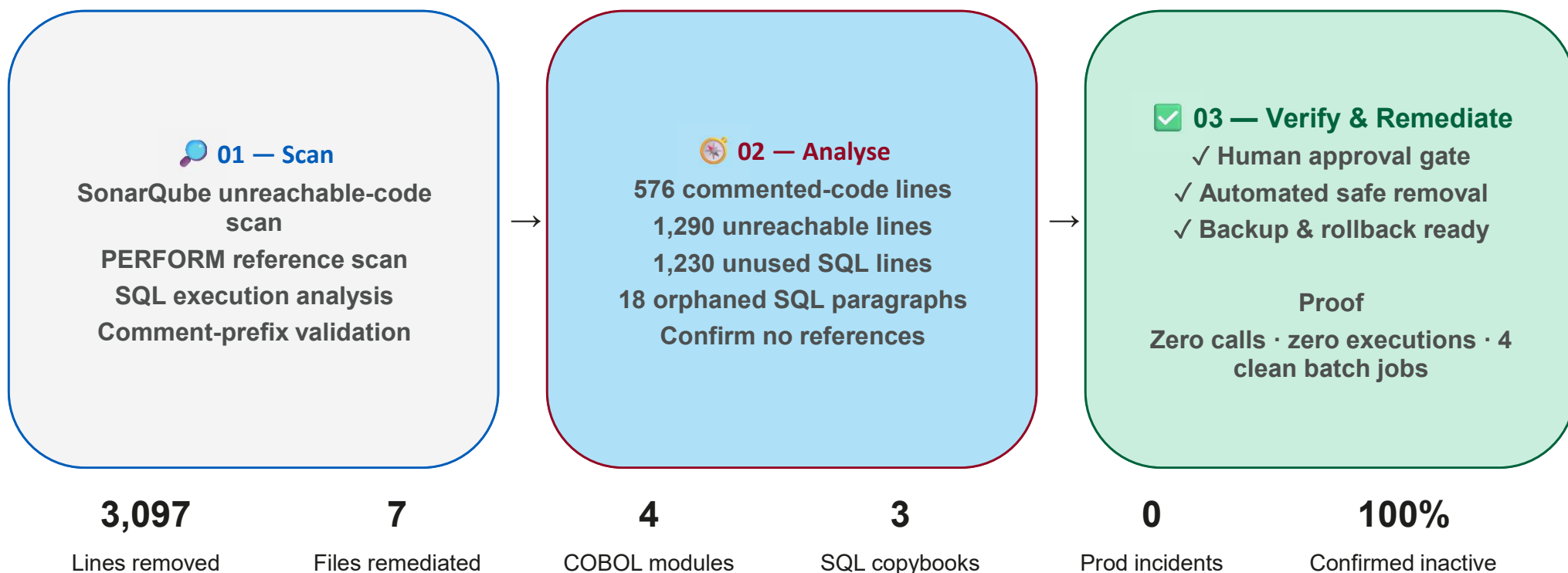
- Developer manually searches for dead code
- Individual knowledge decides what's safe
- Traceability evidence assembled manually across tools

✓ After — AI + Human Control

- AI analyses all modules systematically
- HUMAN approves each recommendation
- AI generates change with full traceability
- HUMAN reviews the final modification

Dead Code Agent Cleans 14+ Years of Technical Debt

3,097 lines · 7 production-critical files · 4 COBOL modules · 3 SQL copybooks · zero production incidents



AGENT #3

Lineage Agent

Turning legacy mainframe artefacts into traceable data lineage

- ? “Where does this business data come from?”
- ? “Which COBOL program updates this table?”
- ? “Which DB2 table contains this information?”
- ? “What business rule causes this value to change?”

Technical Artefacts

Programs, copybooks, feeds, DB2 tables



AI-Assisted Extraction

Dependencies, data flows, transformations



Context Files (Markdown)

Data lineage, traceability, field mappings



Knowledge Space

AI Query Assistants · Natural language Q&A

From Code to Conversations

AI helps developers understand the COBOL application without manually searching across 13+ modules

✗ Before

- Navigate source code manually across 13+ modules
- Specialists inspect program logic, dependencies
- Knowledge stays local — depends on individuals
- Impact analysis requires deep investigation
- Searching COBOL, copybooks, SQL, tracing programs

✓ After

- Ask in natural language — get grounded answers
- Stakeholders query AI assistants for behaviour
- Knowledge becomes reusable across teams
- Context files make discovery repeatable at scale
- Field-level mappings deliver traceable data lineage

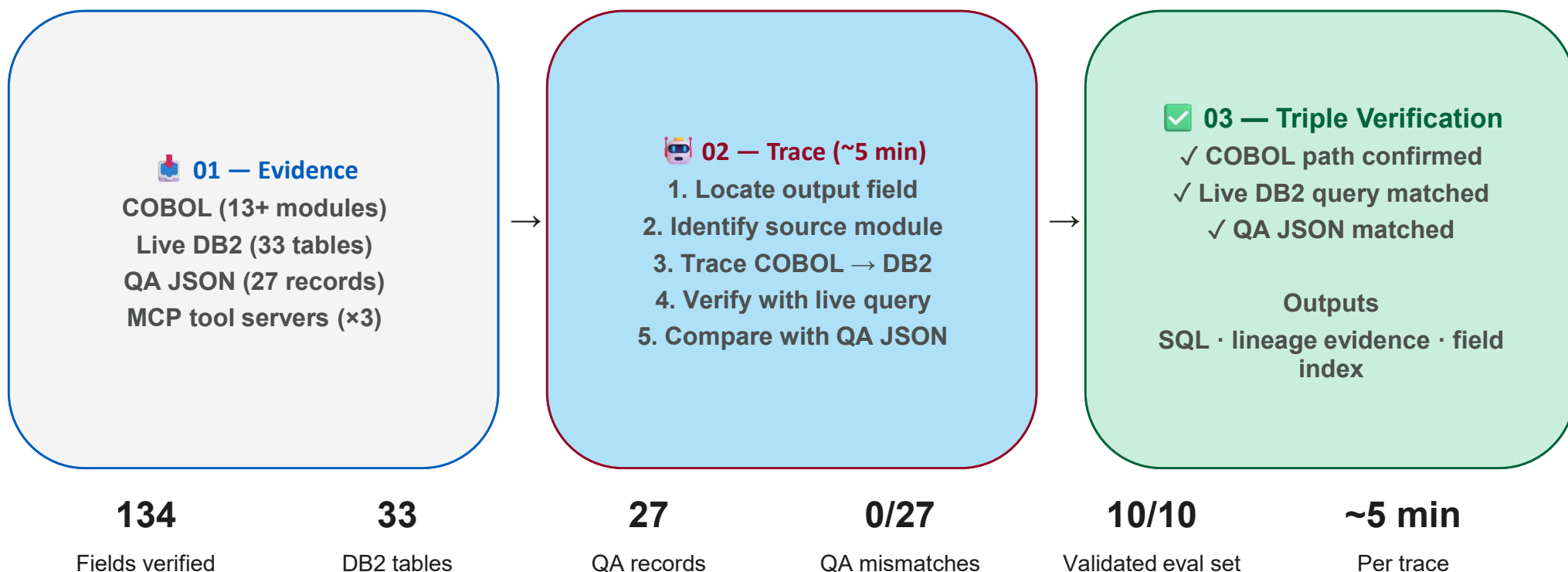
End-to-End Traceability

Input Context → Code Logic → Transformation → Output Context — every field traceable from DB2 source to JSON output

AI Agent Traces Legacy COBOL → Modern Data Lineage



134 fields · 33 DB2 tables · static COBOL analysis · verified against live DB2 and 27 QA records





EVALUATION

Building an Agent Is Half the Job

“Knowing whether you can trust its output is the other half.”



↔ Human-governed loop: review traces → approve prompt/evaluator changes → re-run

EVALUATORS

CODE STRATEGY (5)	accuracy · completeness · hallucination · CVT · SQL
TRAJECTORY (2)	response completeness · branching awareness
DB2 OUTCOME (3)	column · CVT · rule live verification

PROMPT JOURNEY (same validation set)

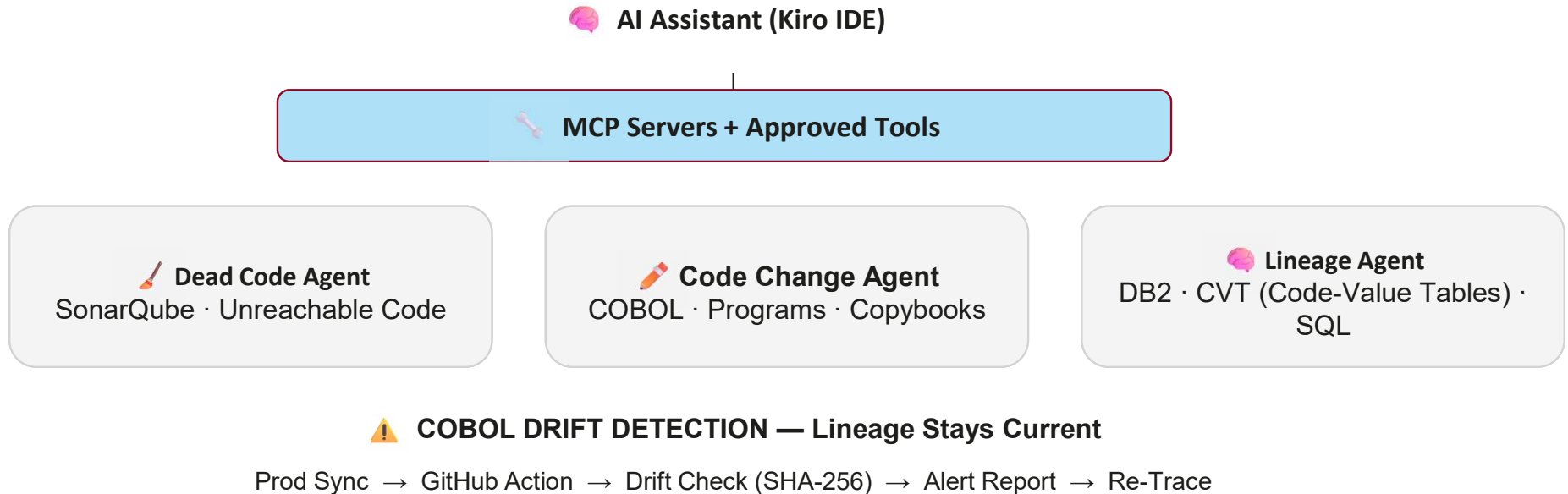
Baseline	5/10
+ Efficiency rules	7/10
Over-simplified prompt	4/10
+ Hard constraints & output spec	10/10



ARCHITECTURE

MCP: Connecting AI to the Tools

The assistant calls governed MCP servers that expose approved tools and data



LESSONS



What I Learned



Start With What You Know

"Start with a problem you already understand."



Build Once, Reuse Many

Prompts and output structures designed to scale.



Treat Complexity as Value

Legacy logic reframed as knowledge to preserve.



Measure Everything

"If you can't measure the output, you can't trust it."



AI Amplifies Expertise

"The AI didn't replace me. It made my expertise more valuable."



The Mainframe Contains Decades of Business Knowledge

AI makes that knowledge easier to access, understand, and maintain.



Don't Leave the Mainframe.

Bring AI to It.

"The best time to start was yesterday. The second best time is now."

Start with a problem you understand.
Learn one language. Try one API.
Build one tool, then one agent.
Evaluate every output. Keep learning.

Your mainframe experience isn't obsolete.
AI makes it more valuable.

KIRO · PYTHON · MCP · ARIZE PHOENIX · OPENTELEMETRY · SONARQUBE · GITHUB ACTIONS · DB2 · COBOL